

ส่วนที่ 1: หลักการออกแบบ

1. อธิบายว่าใช้ SOLID ส่วนไหนบ้างและเกี่ยวข้องกับ Entity อย่างไร

สำหรับ lab08 ใช้หลัก SOLID เพื่อแบ่งหน้าที่ของแต่ละส่วนให้ชัดเจน ทำให้โค้ดสามารถดูแลแก้ไข และขยายระบบได้ง่ายขึ้น โดยมีรายละเอียดดังนี้

1.1 Single Responsibility Principle (SRP) คือ แต่ละคลาสควรมีหน้าที่รับผิดชอบหลักเพียงอย่างเดียว ใน lab นี้ Product, ProductDetail และ Review แยกเป็น Entity คนละตัว เนื่องจากแต่ละ Entity มีหน้าที่แตกต่างกัน

- Product รับผิดชอบข้อมูลหลักของสินค้า เช่น ชื่อสินค้า ราคา หมวดหมู่ และจำนวนสินค้า
 - ProductDetail รับผิดชอบข้อมูลรายละเอียดเพิ่มเติม เช่น น้ำหนัก การรับประกัน และประเทศที่ผลิต
 - Review รับผิดชอบข้อมูลความคิดเห็นของผู้ใช้งาน เช่น คะแนนและข้อความรีวิว
- การแยก ProductDetail ออกจาก Product ทำให้ข้อมูลแต่ละส่วนมีความรับผิดชอบชัดเจน และสอดคล้องกับความสัมพันธ์แบบ 1:1

1.2 Open/Closed Principle (OCP) หมายถึง ระบบควรสามารถเพิ่มความสามารถใหม่ได้โดยไม่ต้องแก้ไขโค้ดเดิมที่ทำงานอยู่ ในส่วนของส่วนลด ระบบใช้ DiscountStrategy เป็น Interface แล้วสร้าง Strategy แยกกัน เช่น

- NoDiscountStrategy
- MemberDiscountStrategy
- SeasonalSaleStrategy

หากต้องการเพิ่มส่วนลดประเภทใหม่ เช่น StudentDiscountStrategy สามารถสร้างคลาสใหม่ที่ Implement DiscountStrategy โดยไม่จำเป็นต้องแก้ไขวิธีคำนวณของ Strategy เดิม

1.3 Liskov Substitution Principle (LSP) หมายถึง Object ของคลาสลูกสามารถนำมาใช้แทน Object ของคลาสแม่หรือ Interface ได้โดยไม่ทำให้โปรแกรมทำงานผิดพลาด สำหรับ lab นี้ Strategy แต่ละตัว Implement DiscountStrategy เหมือนกัน ดังนั้น DiscountContext สามารถรับ Strategy ใดก็ได้ เช่น MemberDiscountStrategy หรือ SeasonalSaleStrategy และสามารถเรียกใช้เมธอดคำนวณส่วนลดในรูปแบบเดียวกัน

1.4 Interface Segregation Principle (ISP) คือไม่ควรบังคับให้คลาสหนึ่งต้องพึ่งพา Method ที่ไม่จำเป็น ในระบบมี Repository แยกตาม Entity เช่น

- ProductRepository
- ProductDetailRepository
- ReviewRepository

แต่ละ Repository รับผิดชอบข้อมูลของ Entity ของตนเอง ทำให้ Service ไม่ต้องพึ่งพา Interface ที่มี Method ที่ไม่เกี่ยวข้องกับงานของตัวเอง

1.5 Dependency Inversion Principle (DIP) คือส่วนที่มีหน้าที่ทำงานหลักไม่ควรขึ้นกับรายละเอียดการทำงานโดยตรง แต่ควรขึ้นกับ Abstraction ตัวอย่างเช่น ProductService รับ ProductRepository ผ่าน Constructor

```
public ProductService(ProductRepository productRepository) {  
    this.productRepository = productRepository;  
}
```

ดังนั้น ProductService ไม่จำเป็นต้องสร้าง Repository เอง ทำให้สามารถเปลี่ยนแปลง Implementation หรือทดสอบ Service ได้ง่ายขึ้น

2. อธิบายความแตกต่างของ 1:1 กับ 1:N — ใช้กรณีไหนควรใช้แบบไหน

ในระบบนี้มีความสัมพันธ์หลัก 2 รูปแบบ คือ 1:1 และ 1:N

2.1 ความสัมพันธ์แบบ 1:1

ความสัมพันธ์แบบ One-to-One (1:1) หมายถึง Entity หนึ่งรายการสัมพันธ์กับอีก Entity หนึ่งรายการเพียงรายการเดียว โดยในระบบใช้

Product 1 ————— 1 ProductDetail

ตัวอย่างเช่น

Product

ID: 1

Name: iPhone 15 Pro

↓ 1:1

ProductDetail

Description: Smartphone

Warranty: 1 Year

Weight: 0.187 kg

กรณีนี้ Product หนึ่งรายการมีรายละเอียดเพิ่มเติมหนึ่งชุด และรายละเอียดหนึ่งชุดเป็นของ Product เพียงหนึ่งรายการ

สำหรับ Java ใช้

```
@OneToOne(cascade = CascadeType.ALL)
```

```
@JoinColumn(name = "detail_id")
```

```
private ProductDetail detail;
```

และควรใช้ 1:1 เมื่อข้อมูลสองส่วนมีความสัมพันธ์แบบ หนึ่งต่อหนึ่งจริง ๆ เช่น

Product → ProductDetail

User → UserProfile

Employee → EmployeeProfile

เหมาะสำหรับกรณีที่ต้องการแยกข้อมูลออกเป็นคนละ Entity แต่ยังต้องการให้แต่ละรายการจับคู่กันแบบหนึ่งต่อหนึ่ง

2.2 ความสัมพันธ์แบบ 1:N

ความสัมพันธ์แบบ One-to-Many (1:N) หมายถึง Entity หนึ่งรายการสามารถมี Entity อีกประเภทหนึ่งได้หลายรายการ โดยในระบบใช้

Product 1 ————— N Review

เช่น

Product: iPhone 15 Pro

|———— Review 1: คะแนน 5
|———— Review 2: คะแนน 4
|———— Review 3: คะแนน 5

ใน Product จึงเก็บ Review เป็น List

```
@OneToMany(  
mappedBy = "product",  
cascade = CascadeType.ALL  
)  
private List<Review> reviews;
```

ส่วนฝั่ง Review จะมี Product เพียงหนึ่งตัว

```
@ManyToOne  
@JoinColumn(name = "product_id")  
private Product product;
```

ควรใช้ 1:N เมื่อ Entity หลักหนึ่งรายการสามารถมีข้อมูลประเภทเดียวกันได้หลายรายการ เช่น

Product → Reviews

Department → Employees

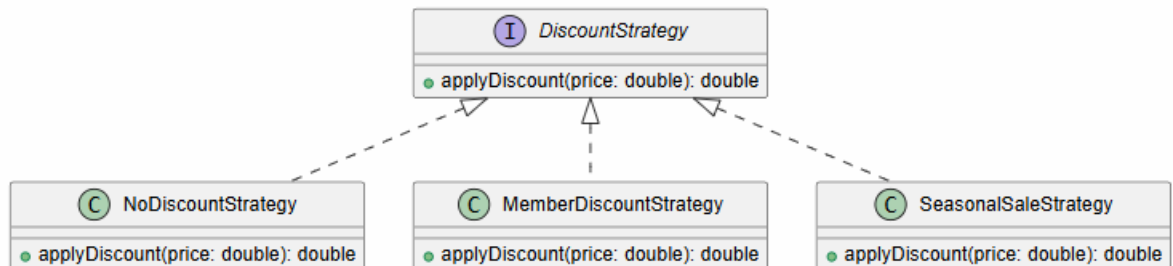
Customer → Orders

Post → Comments

ดังนั้นใน lab นี้ Product กับ Review ต้องใช้ 1:N เพราะสินค้าหนึ่งชิ้นสามารถมีผู้ใช้งานหลายคนมารีวิวได้

3. อธิบาย Strategy Pattern สำหรับคำนวณส่วนลด

ระบบใช้ Strategy Pattern เพื่อแยกวิธีคำนวณส่วนลดแต่ละประเภทออกจากกัน โดยมีโครงสร้างคือ



Interface DiscountStrategy กำหนด Method สำหรับคำนวณราคาหลังหักส่วนลด

```
public interface DiscountStrategy {  
    double applyDiscount(double price);  
}
```

แต่ละ Strategy มีวิธีคำนวณต่างกัน เช่น

No Discount

ราคา = 1,000

ราคาสุทธิ = 1,000 บาท

Member 10%

ราคา = 1,000

ราคาสุทธิ = 900 บาท

Seasonal 20%

ราคา = 1,000

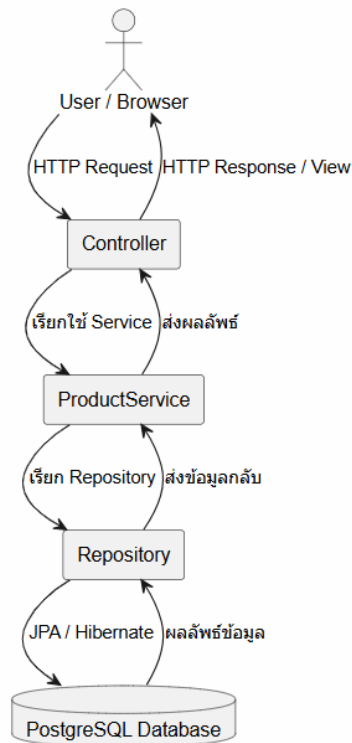
ราคาสุทธิ = 800 บาท

จากนั้น DiscountContext ทำหน้าที่เลือกและเรียกใช้ Strategy ที่เหมาะสม

ข้อดีคือ หากต้องการเพิ่มประเภทส่วนลดใหม่ ไม่จำเป็นต้องนำ if-else จำนวนมากไปใส่ใน Product หรือ ProductService แต่สามารถสร้าง Strategy ใหม่แยกออกมาได้ ทำให้ระบบขยายได้ง่าย และสอดคล้องกับ OCP

4. อธิบาย Execution Flow ตั้งแต่ HTTP Request → DB

การทำงานของระบบแบ่งออกเป็น Layer เพื่อให้แต่ละส่วนมีหน้าที่ชัดเจน



ตัวอย่างกรณีผู้ใช้กด เพิ่มสินค้า

Step 1: Browser

ผู้ใช้กรอกข้อมูล Product, ProductDetail และ Review แล้วกดบันทึกสินค้า
Form ส่ง

POST /products/save

Step 2: Controller

ProductController รับ HTTP Request

@PostMapping("/save")

```
public String save(@ModelAttribute Product product) {  
    productService.save(product);  
    return "redirect:/products";  
}
```

Controller มีหน้าที่รับ Request และส่งต่อให้ Service ไม่ควรนำ Business Logic จำนวนมากมาใส่ไว้ใน Controller

Step 3: Service

ProductService ทำหน้าที่จัดการ Business Logic เช่น การเชื่อม Review เข้ากับ Product

```
review.setProduct(product);
```

จากนั้นเรียก Repository เพื่อบันทึกข้อมูล

```
productRepository.save(product);
```

Step 4: Repository

Repository ทำหน้าที่ติดต่อกับ Database ผ่าน Spring Data JPA

```
public interface ProductRepository  
    extends JpaRepository<Product, Long> {  
}
```

JPA/Hibernate จะสร้าง SQL ที่จำเป็นให้โดยอัตโนมัติ

Step 5: PostgreSQL

ข้อมูลถูกบันทึกลง PostgreSQL ในตาราง

- products
- product_details
- reviews

โดยมี Foreign Key เชื่อมความสัมพันธ์

products



ส่วนที่ 2: Codeและคำอธิบาย

1. Entity Product

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String category;

    private String brand;

    private Integer stock;

    private Double price;

    private String discountType;

    /*
     * ราคาหลังหักส่วนลด
     * ไม่สร้าง column ใน database
     */
    @Transient
    private Double discountedPrice;
```

```
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(
    name = "detail_id",
    referencedColumnName = "id"
)
private ProductDetail detail;

// =====
// 1:N Product -> Review
// =====

@OneToMany(
    mappedBy = "product",
    cascade = CascadeType.ALL
)
private List<Review> reviews = new ArrayList<>();
```

Annotation ที่สำคัญ

- @Entity
บอก Spring Data JPA ว่าคลาสนี้เป็น Entity ที่ต้องนำไปสร้างเป็นตารางใน Database
- @Table(name = "products")
กำหนดชื่อ Table ใน Database เป็น products
- @Id
กำหนด Field id ให้เป็น Primary Key
- @GeneratedValue(strategy = GenerationType.IDENTITY)
ให้ Database สร้างค่า ID ให้อัตโนมัติ
- @OneToOne
กำหนดความสัมพันธ์แบบ 1:1 ระหว่าง Product และ ProductDetail
- @JoinColumn(name = "detail_id")
กำหนด Foreign Key detail_id ในตาราง products
- @OneToMany
กำหนดว่า Product หนึ่งรายการสามารถมี Review ได้หลายรายการ
- mappedBy = "product"
บอกว่าเจ้าของความสัมพันธ์อยู่ที่ Field product ใน Entity Review
- cascade = CascadeType.ALL
การดำเนินการกับ Product เช่น Save/Delete สามารถส่งผลต่อ Entity ที่เกี่ยวข้องตาม Cascade ที่กำหนด
- @Transient
บอก JPA ว่า Field/Property นี้ไม่ต้องสร้างเป็น Column ใน Database เช่น discountedPrice ซึ่งเป็นค่าที่คำนวณจากราคาสินค้า

2. Entity ProductDetail

```
@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String description;

    private String warranty;

    private Double weight;

    private String dimensions;

    private String manufacturedCountry;

    // Inverse side of 1:1 relationship
    @OneToOne(mappedBy = "detail")
    private Product product;
```

Annotation สำคัญ

- @Entity ใช้กำหนดว่าเป็น Entity
- @Table กำหนดชื่อ Table
- @Id และ @GeneratedValue ใช้กำหนด Primary Key และสร้าง ID อัตโนมัติ
- ส่วน
- @OneToOne(mappedBy = "detail") หมายถึง ProductDetail เป็นอีกฝั่งของความสัมพันธ์ 1:1 และความสัมพันธถูกควบคุมโดย Field detail ใน Product ดังนั้น

Product.detail



ProductDetail.product

จึงเชื่อมกันเป็นความสัมพันธ์แบบ 1:1

3. Entity Review

```
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String reviewer;

    private Integer rating;

    private String comment;

    private LocalDate reviewDate;

    // =====
    // N:1 Review -> Product
    // =====

    @ManyToOne
    @JoinColumn(name = "product_id")
    private Product product;
```

Annotation สำคัญ

- @Entity กำหนดให้ Review เป็น JPA Entity
- @Table(name = "reviews") กำหนดชื่อ Table เป็น reviews
- @Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
กำหนด id เป็น Primary Key และให้ Database สร้าง ID อัตโนมัติ

- @ManyToOne

@JoinColumn(name = "product_id")

private Product product;

หมายถึง Review หลายรายการสามารถเป็นของ Product เดียวกันได้

ตัวอย่าง

Product ID = 1

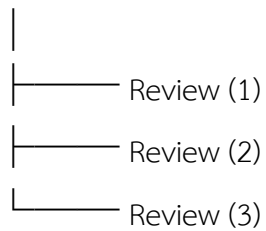
Review ID 1 → product_id = 1

Review ID 2 → product_id = 1

Review ID 3 → product_id = 1

จึงเกิดความสัมพันธ์

Product (1)



และ product_id จะเป็น Foreign Key ในตาราง reviews

4. ProductService

ตัวอย่างโครงสร้าง Service

```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    // Constructor Injection
    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    // =====
    // READ ALL
    // =====

    public List<Product> findAll() {

        List<Product> products = productRepository.findAll();

        for (Product product : products) {
            product.setDiscountedPrice(
                calculateFinalPrice(product)
            );
        }

        return products;
    }

    public Product findById(Long id) {

        return productRepository.findById(id)
            .orElseThrow(
                () -> new RuntimeException(
                    "Product not found: " + id
                )
            );
    }
}
```

```

public Product save(Product product) {

    // ProductDetail
    if (product.getDetail() != null) {
        product.getDetail().setProduct(product);
    }

    // Review
    if (product.getReviews() != null) {
        for (Review review : product.getReviews()) {
            if (review.getReviewer() != null
                && !review.getReviewer().isBlank()) {
                review.setProduct(product);

                if (review.getReviewDate() == null) {
                    review.setReviewDate(LocalDate.now());
                }
            }
        }
    }

    return productRepository.save(product);
}

public void delete(Long id) {
    productRepository.deleteById(id);
}

```

การใช้ Constructor Injection ช่วยลดการผูกกันแน่นระหว่าง Class และสอดคล้องกับหลัก DIP ทำให้สามารถทดสอบและเปลี่ยน Dependency ได้ง่ายขึ้น อีกทั้งยังทำให้เห็น Dependency ของแต่ละ Class อย่างชัดเจน และ Spring สามารถจัดการ Object และ Dependency ให้โดยอัตโนมัติ

5. ProductController

```
@Controller
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    // Constructor Injection
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public String list(Model model) {

        model.addAttribute(
            attribute: "products",
            productService.findAll()
        );

        return "products/list";
    }
}
```

```
@GetMapping("/add")
public String add(Model model) {

    Product product = new Product();

    // สร้าง ProductDetail สำหรับ 1:1
    product.setDetail(new ProductDetail());

    // สร้าง Review ตัวแรกสำหรับ reviews[0]
    product.getReviews().add(new Review());

    model.addAttribute(attributeName: "product", product);

    return "products/add";
}

@PostMapping("/save")
public String save(@ModelAttribute Product product) {

    productService.save(product);

    return "redirect:/products";
}
```

```

@GetMapping("/edit/{id}")
public String edit(
    @PathVariable Long id,
    Model model
) {

    Product product =
        productService.findById(id);

    model.addAttribute(
        attributeName: "product",
        product
    );

    return "products/edit";
}

```

```

@PostMapping("/update/{id}")
public String update(
    @PathVariable Long id,
    @ModelAttribute Product product
) {

    productService.update(id, product);

    return "redirect:/products";
}

```

```

@GetMapping("/delete/{id}")
public String deletePage(
    @PathVariable Long id,
    Model model
) {

    Product product =
        productService.findById(id);

    model.addAttribute(
        attributeName: "product",
        product
    );

    return "products/delete";
}

```

```

@PostMapping("/delete/{id}")
public String delete(
    @PathVariable Long id
) {

    productService.delete(id);

    return "redirect:/products";
}

```

Annotation ของ Controller

- @Controller

บอก Spring ว่าคลาสนี้เป็น Controller สำหรับรับ HTTP Request และส่งข้อมูลไปยัง View

- @RequestMapping("/products")

กำหนด URL หลักของ Controller เป็น

/products

ดังนั้น

@GetMapping

จะเป็น

GET /products

@GetMapping("/add")

จะเป็น

GET /products/add

- @GetMapping

ใช้รับ HTTP GET Request เช่น การเปิดหน้าเว็บ

- @PostMapping

ใช้รับ HTTP POST Request เช่น การส่ง Form เพื่อบันทึกข้อมูล

- @PathVariable

ใช้รับค่าจาก URL เช่น

/products/edit/5

โดยเลข 5 จะถูกส่งเข้า

@PathVariable Long id

- @ModelAttribute

ใช้รับข้อมูลจาก HTML Form แล้วแปลงเป็น Object Product

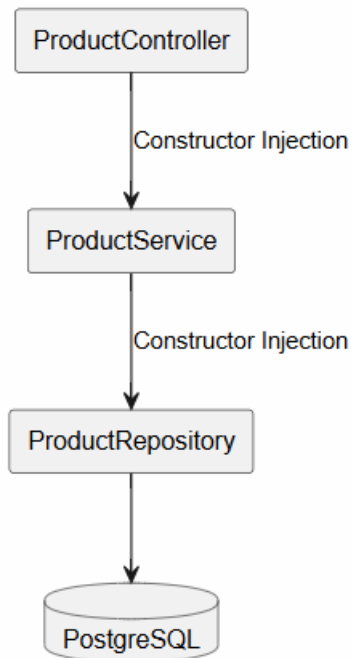
6. Constructor Injection ใน Controller

ใน Controller ใช้รูปแบบเดียวกับ Service

```
private final ProductService productService;

// Constructor Injection
public ProductController(ProductService productService) {
    this.productService = productService;
}
```

Spring จะสร้าง ProductService แล้วส่งเข้ามาให้ ProductController
จึงเกิด Dependency Flow :



ส่วนที่ 3: ภาพหน้าจอ

The screenshot shows a web browser at localhost:8080/products/add. The page has a header with a hamburger menu icon and a 'School' user profile. The main content area is titled '+ เพิ่มสินค้าใหม่' (Add New Product). It contains a form with the following fields:

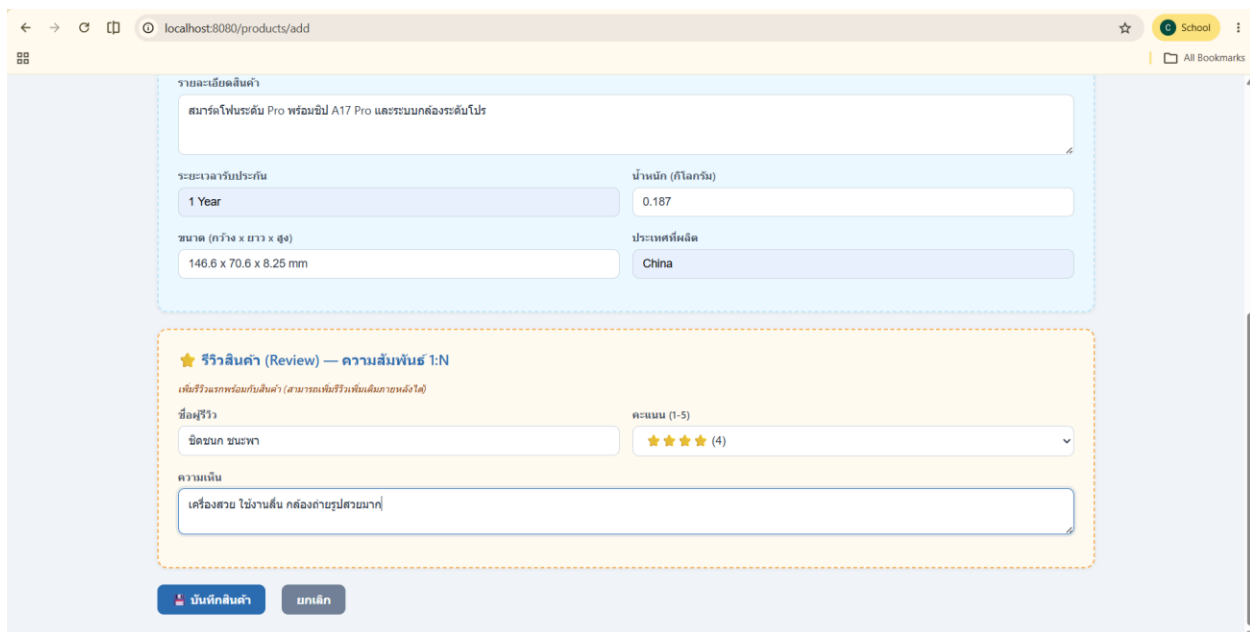
- ข้อมูลสินค้า (Product):**
 - ชื่อสินค้า (Product Name): iPhone 15 Pro (673380033-9 SEC 1)
 - หมวดหมู่ (Category): electronic
 - ยี่ห้อ (Brand): Apple
 - จำนวนในคลัง (Inventory): 10
 - ราคารวมภาษี (Price with tax): 39900
 - ประเภทส่วนลด (Discount type): ส่วนลดสมาชิก (10%)
- ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1:**
 - รายละเอียดสินค้า (Product Detail): Sma
 - ระยะเวลาประกัน (Warranty period): เช่น 1 Year, 6 Months
 - น้ำหนัก (กิโลกรัม) (Weight in kg): เช่น 1.61

ภาพที่ 3.1 Create กรอกข้อมูล Product

This screenshot shows the same form as the previous one, but with additional details filled in:

- ข้อมูลสินค้า (Product):**
 - ราคารวมภาษี (Price with tax): 39900
 - ประเภทส่วนลด (Discount type): ส่วนลดสมาชิก (10%)
- ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1:**
 - รายละเอียดสินค้า (Product Detail): สมาร์ทโฟนระดับ Pro พร้อมชิป A17 Pro และระบบกล้องระดับโปร
 - ระยะเวลาประกัน (Warranty period): 1 Year
 - น้ำหนัก (กิโลกรัม) (Weight in kg): 0.187
 - ขนาด (กว้าง x ยาว x สูง) (Dimensions): 146.6 x 70.6 x 8.25 mm
 - ประเทศที่ผลิต (Country of origin): China
- รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N:**
 - หัวข้อรีวิว (Review title): เช่น สมชาย ใจดี
 - คะแนน (1-5) (Rating): ★★★★★ (5)

ภาพที่ 3.2 Create กรอกข้อมูล ProductDetail



รายละเอียดสินค้า

สมาร์ตโฟนระดับ Pro พร้อมชิป A17 Pro และระบบกล้องระดับโปร

ระยะเวลาประกัน: 1 Year น้ำหนัก (กิโลกรัม): 0.187

ขนาด (กว้าง x ยาว x สูง): 146.6 x 70.6 x 8.25 mm ประเทศที่ผลิต: China

★ รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N

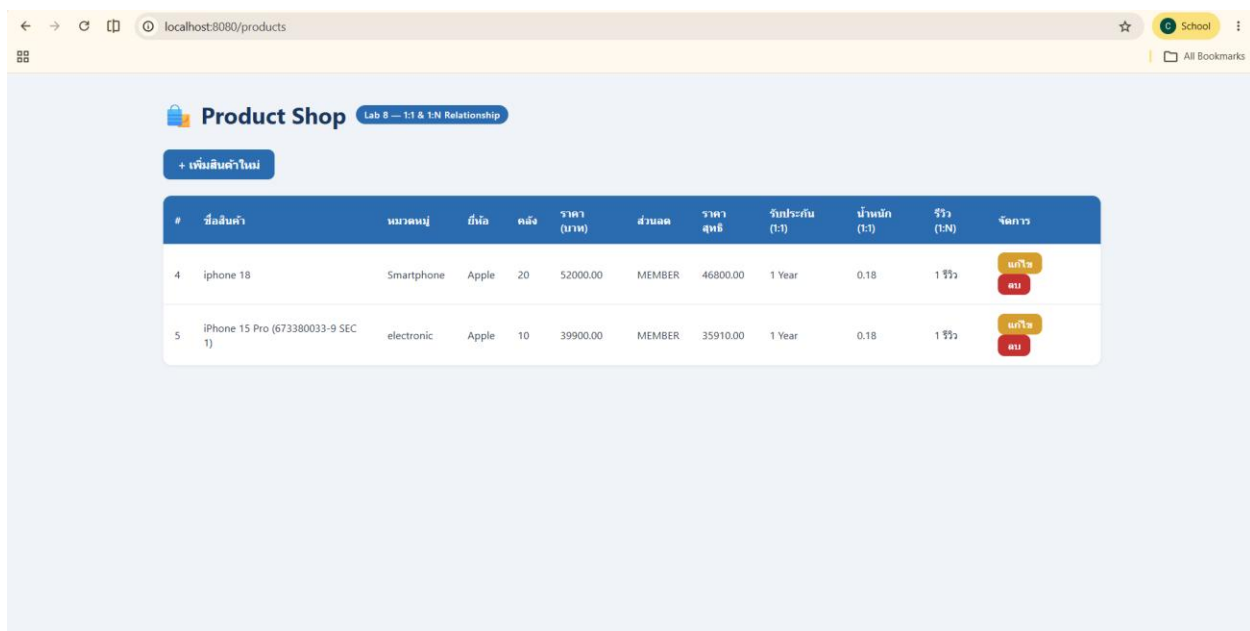
เพื่อรีวิวจากเพื่อนกับสินค้า (สามารถเห็นรีวิวที่เห็นสินค้าของคุณได้)

ชื่อผู้รีวิว: ชิดชนก ขนะพา คะแนน (1-5): ★★★★★ (4)

ความเห็น: เครื่องสวย ใช้งานสั้น คัดลอกง่ายสะดวก

บันทึกสินค้า ยกเลิก

ภาพที่ 3.3 Create กรอกข้อมูล Review



Product Shop Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	แบรนด์	คงเหลือ	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	ประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
4	iPhone 18	Smartphone	Apple	20	52000.00	MEMBER	46800.00	1 Year	0.18	1 รีวิว	แก้ไข ลบ
5	iPhone 15 Pro (673380033-9 SEC 1)	electronic	Apple	10	39900.00	MEMBER	35910.00	1 Year	0.18	1 รีวิว	แก้ไข ลบ

ภาพที่ 3.4 Read รายการสินค้าในตาราง

localhost:8080/products/edit/5

iPhone 15 Pro (673380033-9 SEC 1) | electronic

ยี่ห้อ: Apple | จำนวนในคลัง: 10

ราคาปกติ (บาท): 39900.0 | ประเภทส่วนลด: ส่วนลดสมาชิก (10%)

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า: สมาร์ทโฟนระดับ Pro พร้อมชิป A17 Pro และระบบกล้องระดับโปร

ระยะเวลาประกัน: 1 Year | น้ำหนัก (กิโลกรัม): 0.18

ขนาด: 146.6 x 70.6 x 8.25 mm | ประเทศที่ผลิต: China

บันทึกการแก้ไข | ยกเลิก

ภาพที่ 3.5 Update หน้าฟอร์มแก้ไข

localhost:8080/products/delete/4

ยืนยันการลบสินค้า

คุณต้องการลบสินค้านี้ใช่หรือไม่? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า: iphone 18

หมวดหมู่: Smartphone

ยี่ห้อ: Apple

ราคา: 52000.00 บาท

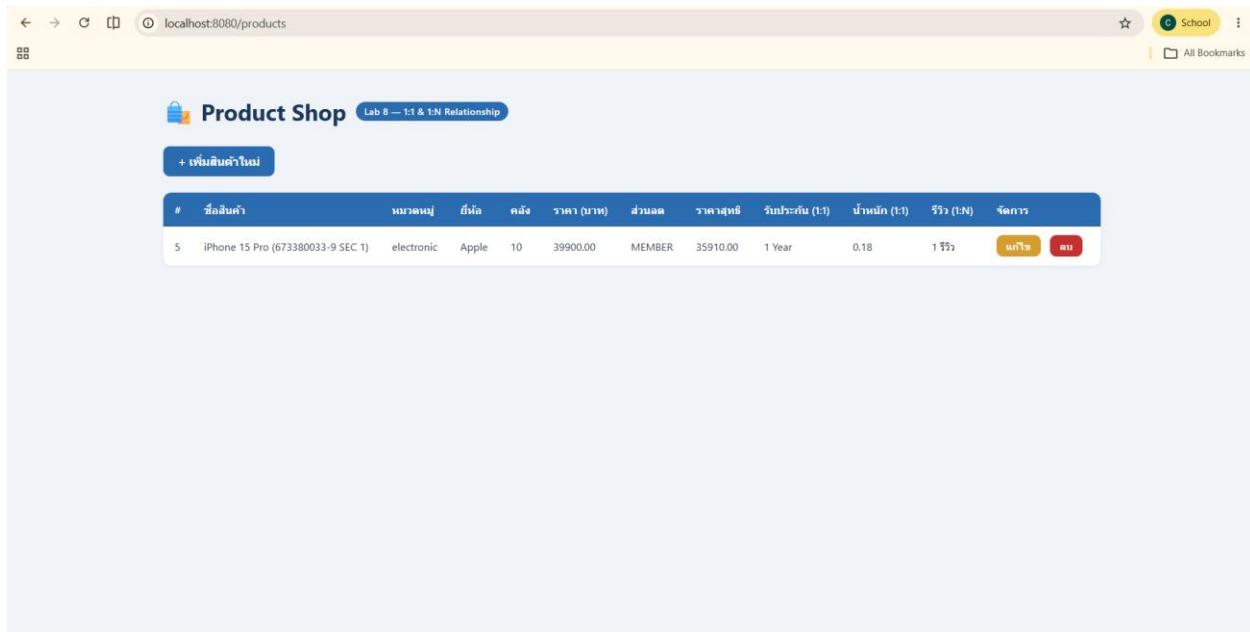
รายละเอียด: Smartphone

รับประกัน: 1 Year

⚠️ ข้อมูลเสริม (ProductDetail) จะถูกลบพร้อมกันด้วย เนื่องจาก CascadeType:ALL

ยืนยันลบ | ยกเลิก

ภาพที่ 3.6 Delete ยืนยันลบ



ภาพที่ 3.7 Delete หลังลบ

Data Output Messages Notifications									
Showing rows: 1 to 1 Page No: 1 of 1									
	id [PK] bigint	brand character varying (255)	category character varying (255)	discount_type character varying (255)	name character varying (255)	price double precision	stock integer	detail_id bigint	
1	5	Apple	electronic	MEMBER	iPhone 15 Pro (673380033-9 SEC...	39900	10	5	

ภาพที่ 3.8 Database แสดงตาราง products

Data Output Messages Notifications						
Showing rows: 1 to 1 Page No: 1 of						
	id [PK] bigint	description character varying (255)	dimensions character varying (255)	manufactured_country character varying (255)	warranty character varying (255)	weight double precision
1	5	สมาร์ตโฟนระดับ Pro พร้อมชิป A17 Pro และระบบกล้องระดับ...	146.6 x 70.6 x 8.25 mm	China	1 Year	0.18

ภาพที่ 3.9 Database แสดงตาราง product_details

Data Output Messages Notifications						
Showing rows: 1 to 1						
	id [PK] bigint	comment character varying (255)	rating integer	review_date date	reviewer character varying (255)	product_id bigint
1	5	เครื่องสวย ใช้งานลื่น กล้องถ่ายรูปสวยม...	4	2026-09-09	ชิตชนก ชนะพา	5

ภาพที่ 3.10 Database แสดงตาราง reviews